Listas

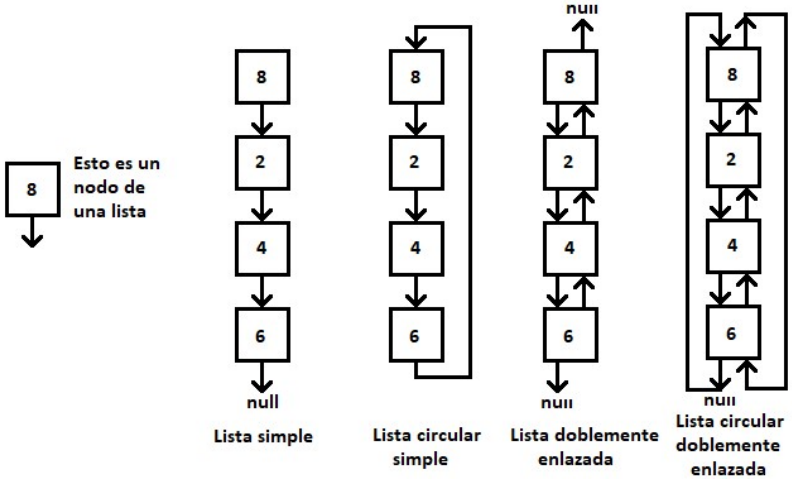
Si seguimos avanzando en complejidad con estructuras de datos, nos encontramos con las listas. Las listas de datos son estructuras que permiten organizar y gestionar información de manera eficiente. Estas nos facilitan la inserción y eliminación dinámica de elementos. Existen diversas variaciones de listas enlazadas, cada una con características que las hacen más adecuadas para ciertas aplicaciones. La idea de contar con una lista a diferencia de las pilas y colas es que estas últimas tienen un tamaño estático y las listas tienen un tamaño dinámico, en teoría su máximo sería la disponibilidad de memoria RAM del sistema.

En esta etapa veremos cuatro tipos de listas, las cuales son muy similares entre ellas con muy pequeñas variaciones. Interpretemos a las listas como un conjunto de datos, en donde cada dato se enlaza con otro de tal manera que estos enlaces van formando la lista, cada uno de estos datos los llamaremos nodo, y como hicimos en las estructuras anteriores vamos a trabajar con nodos de datos enteros, así podemos simplificar su estudio.

Aplicaciones de los distintos tipos de listas

- Manejo de memoria dinámica
- Implementación de colas y pilas
- Almacenamiento de secuencias en las que se requiere un acceso secuencial
- Juegos que requieren un recorrido circular (p.ej. ruleta)
- Simuladores de sistemas de cola circular, como planificadores de procesos en sistemas operativos
- Implementación de estructuras como listas de navegación (atrás y adelante)
- Manipulación de secuencias en ambas direcciones (editores de texto, navegadores)
- Sistemas de planificación de procesos que requieren recorridos circulares bidireccionales
- Aplicaciones que requieren un manejo eficiente del acceso hacia adelante y hacia atrás, en ciclos continuos

Veamos gráficamente los distintos tipos de listas que vamos a trabajar



Lista simple

Este tipo de lista, es la más sencilla de las 4, cuenta con nodos enlazados uno a continuación de otro. Vamos a contar con la dirección de memoria del primer nodo, y a través de este podemos ir avanzando en la lista hacia adelante para recorrer el resto de los nodos. Es importante marcar que no podemos perder el puntero al primer nodo, ya que estaríamos perdiendo el acceso a toda la lista. Esta lista tiene la característica que ultimo nodo siempre estará apuntando a ***null***, y ya en su lógica podemos notar que se pueden ir agregando nodo a medida que se necesite sin tener en cuenta ningún tipo de limite predefinido como por ejemplo el tamaño de un vector.

Características

- Nodo inicial (cabeza o head): Es el primer nodo de la lista.
- Último nodo: El último nodo de la lista tiene un puntero que apunta a null (nulo).
- Acceso secuencial: Los nodos solo pueden ser accedidos secuencialmente, comenzando desde el primer nodo.

Codificación

```
typedef struct nodo
{
    int dato;
    struct nodo *siguiente;
} tnodo;
```

Para crear este tipo de listas solamente necesitamos la estructura del nodo, el cual está compuesto por una variable entera que almacena el dato y por un puntero que va a contener una dirección de memoria hacia otro del mismo tipo que estamos definiendo. De esta manera con ese puntero vamos a ir estableciendo el enlace entre nodo y nodo.

Para poder manipular la lista, necesitamos una función que nos cree un nodo y lo ponga a disponibilidad para enlazar. Veamos como lo podemos codificar:

```
tnodo* crearNodoSimple(int dato)
{
    tnodo *nuevo=(tnodo*)malloc(sizeof(tnodo));
    if(nuevo!=NULL)
    {
        nuevo->dato=dato;
        nuevo->siguiente=NULL;
    }
    else{
        printf("No se pudo crear nodo\n");
    }
}
```

Esta función crear nodo, trabaja internamente con la función malloc para realizar una reserva de memoria. Al reservar memoria debemos verificar que se realizó con éxito y a partir de ahí podemos asignar el dato para devolver el nodo creado.

Enlazando los datos

Al estar trabajando con una lista simple nuestro concepto es ir agregando datos al final de la lista. Si partimos con la idea de una lista encadenada deberíamos avanzar en esta lista hasta llegar al último nodo, el cual apunta a null, y ahí proceder a insertar el nodo.

```
void agregarNodoAlFinal(tnodo * lista,int dato)
{
    while(lista->siguiente!=NULL)
    {
        lista=lista->siguiente;
    }
    lista->siguiente=crearNodoSimple(dato);
}
```

De manera similiar, usando esta lógica podemos hacer una función que recorra la lista y nos vaya mostrando los datos de la misma. El concepto o idea general para recorrer o avanzar en la lista es utilizar un while e ir cambiando nodo a nodo.

```
void mostrarListaSimple(tnodo *lista)
{
    while(lista!=NULL)
    {
        printf("%d\n",lista->dato);
        lista=lista->siguiente;
    }
}
```

Ejemplo práctico de aplicación en un main

```
int main()
{
    tnodo *lista=crearNodoSimple(3);
    agregarNodoAlFinal(lista,8);
    agregarNodoAlFinal(lista,5);
    agregarNodoAlFinal(lista,1);
    agregarNodoAlFinal(lista,9);
    agregarNodoAlFinal(lista,11);
    mostrarListaSimple(lista);
    return 0;
}
```

Lista circular

Las listas circulares son similares a las listas simples con la diferencia que el ultimo nodo en vez de apuntar a null apunta al primer nodo. De esta manera se obtiene una lista circular en la que se puede ir avanzando indefinidamente. Debemos tener en cuenta que para manipular estas listas necesitamos tomar un punto de referencia del inicio para no caer en un bucle infinito de recorridas. Otro detalle a tener en cuenta es que estas listas no cambian el código con respecto a cómo se crean e inicializan los nodos; únicamente vemos modificaciones en el agregado de datos y su recorrida.

Recorriendo una lista circular

```
void mostrarListaCircular(tnodo *lista)
{
    tnodo *primero=lista;
    do
    {
        printf("%d\n",lista->dato);
        lista=lista->siguiente;
    }while(lista!=primero);
}
```

Vemos como necesitamos tomar como referencia el puntero de inicio de la lista, vamos avanzando y comparamos hasta llegar al mismo puntero de inicio. Este mismo principio es el que vamos a aplicar para insertar datos en forma circular.

Insertando datos circularmente

```
void agregarNodoAlFinal(tnodo * lista,int dato)
{
    tnodo *nuevo=crearNodoSimple(dato);
    if(lista->siguiente==NULL)
    {
        lista->siguiente=nuevo;
        nuevo->siguiente=lista;
    }
    else
    {
        tnodo *primero=lista;
        while(lista->siguiente!=primero)
        {
            lista=lista->siguiente;
        }
        lista->siguiente=nuevo;
        nuevo->siguiente=primero;
    }

}
```

Con la diferencia que tenemos que verificar si es el primer nodo que se inserta para armar la lista circular. Una vez insertado el primero se procede a avanzar circularmente hasta llegar al último nodo. El ejemplo de aplicación en el main es similar a la lista simple.

Lista Doblemente enlazada

Estas listas agregan un puntero adicional para volver al nodo anterior, así que podemos tomar la estructura de la lista simple y agregar una línea adicional de puntero anterior

```
typedef struct nodo
{
    int dato;
    struct nodo *siguiente;
    struct nodo *anterior;
} tnodo;
```

La función crear nodo para una lista doblemente enlazada también es similar en la que solamente también agregamos el puntero anterior a null:

```
tnodo* crearNodoDoble(int dato)
{
    tnodo* nuevo=(tnodo*)malloc(sizeof(tnodo));
    if(nuevo!=NULL)
    {
        nuevo->dato=dato;
        nuevo->anterior=NULL;
        nuevo->siguiente=NULL;
    }
}
```

Agregando datos

```
void agregarNodoAlFinal(tnodo * lista,int dato)
{
    while(lista->siguiente!=NULL)
    {
        lista=lista->siguiente;
    }
    tnodo*nuevo=crearNodoDoble(dato);
    lista->siguiente=nuevo;
    nuevo->anterior=lista;
}
```

Para agregar datos seguimos la misma lógica aplicada a una lista simple, pero debemos armar los enlaces correspondientes entre el ultimo nodo y el nodo nuevo creado, de tal manera que ambos se vinculen en las dos direcciones.

El procedimiento de mostrar los datos de una lista circular es igual al de una lista simple, pero a partir de esta idea podríamos crear funciones que una vez que llegan al final puedan retroceder y mostrar la lista desde el final hacia el principio.

```
void mostrarListaDoble(tnodo *lista)
{
     while(lista!=NULL)
    {
        printf("%d\n",lista->dato);
        lista=lista->siguiente;
    }
}


int main(){

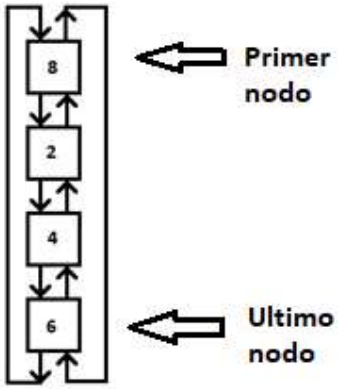
     tnodo *lista=crearNodoDoble(3);
    agregarNodoAlFinal(lista,8);
    agregarNodoAlFinal(lista,5);
    agregarNodoAlFinal(lista,1);
    agregarNodoAlFinal(lista,9);
    agregarNodoAlFinal(lista,11);
    mostrarListaDoble(lista);
    return 0;

}
```

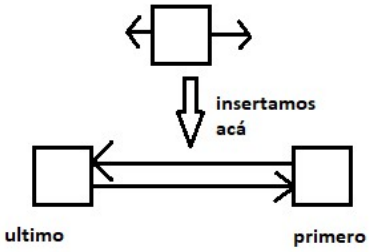
Listas circulares doblemente enlazadas

Llegando a la última de las listas nos encontramos con que es una combinación de la estructura de una lista doble enlazada y la lógica aplicada a una circular, por lo tanto, nuestra estructura para esta lista va a ser igual a la utilizada en las listas doblemente enlazadas. Pero para su recorrido aplicamos la lógica de listar circulares. La función de crear nodo no cambia así que directamente pensemos en cómo podemos ingresar datos en esta lista.

En primer lugar, al ser una lista circular doblemente enlazada el ultimo nodo está ubicado por detrás del primero, vemos el diagrama



Entonces si queremos insertar un nodo en el último lugar lo podemos insertar hacia atrás sobre el primer nodo.



Entonces los enlaces a modificar en este caso van a ser 4, dos del nodo nuevo (anterior y siguiente), uno anterior del nodo primero y el siguiente del nodo último. Veamos el código:

```
void agregarNodoAlFinal(tnodo * lista,int dato)
{
    tnodo*nuevo=crearNodoDoble(dato);
    if(lista->siguiente==NULL)
    {
        lista->siguiente=nuevo;
        lista->anterior=nuevo;
        nuevo->siguiente=lista;
        nuevo->anterior=lista;
    }
    else{
            lista->anterior->siguiente=nuevo;
            nuevo->anterior=lista->anterior;
            lista->anterior=nuevo;
            nuevo->siguiente=lista;
    }
}
```

El recorrido es igual a una lista circular simple:

```
void mostrarListaCircularDoble(tnodo *lista)
{
    tnodo *primero=lista;
    do
    {
        printf("%d\n",lista->dato);
        lista=lista->siguiente;
    }while(lista!=primero);
}
```